

Problem 1:

Carefully read and understand the posted solutions to the **midterm exam**. Identify **one problem from the list below** for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.).

- Q10 Chaotic Potential
- Q13 Greedy Cat Photographing
- Q14 DFA Impossibility
- Q15 SpongeBob vs Squidward
- Q16 Machine Disagreement

Copy or screenshot your solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't take the exam, review this solution by GPT 4o instead (pick one problem and explain the room for improvement).

10. You are observing a highly unstable process unfolding in a chaotic energy field. The process involves two types of particles: maize and blue. In each time unit, **one** of the following reactions must occur at random:

- **Fusion:** Two maize particles combine to form one blue particle.
- **Decay:** One blue particle destabilizes and transforms into one maize particle.

The process *halts* if no more reactions are possible.

- (a) Let b and m be the number of blue and maize particles. **Give a potential function $s(b, m)$ that can be used to prove that this process eventually halts.**

$$s(b, m) = b + m$$

- (b) **Briefly (in 2-3 sentences) explain why your answer in Part a works.**

If fusion occurs then $b+m \rightarrow (m-2)+(b+1) = b+m-1 < b+m$
 If decay occurs then $b+m \rightarrow b-1+m = b+m-1 < b+m$
 Thus, decreases by fixed amount, and lowerbounded by zero since we can't have less than zero b, m .

My solution was deficient because I forgot that the amount of maize particles decreased by one in the decay stage.

I could have fixed with the potential function $3b + 2m$. Since if fusion then $3b + 2m \rightarrow 3(b + 1) + 2(m - 1) = 3b + 2m - 1$ and if decay then $3b + 2m \rightarrow 3(b - 1) + 2(m + 1) = 3b + 2m - 1$

Problem 2: Understanding P, NP, and NP-hardness.**(a)**

Prove that all efficiently decidable languages are efficiently verifiable. In other words, show that every language in P has an efficient verifier.

Proof.

Let $L \in P$ thus there exists an efficient Turing Machine M that decides L in polynomial time.

Let $V(x, c) = M(x)$.

$V(x, c)$ is polynomial time since $M(x)$ is.

If $x \in L$ then $M(x)$ accepts thus $V(x, c)$ accepts.

If $x \notin L$ then $M(x)$ rejects thus $V(x, c)$ rejects for all c since we ignore it. ■

(b)

Prove that every language in NP is decidable. In other words, show that every language in NP has a decider.

Proof.

Let $L \in NP$ thus there exists an efficient verifier V that is bounded by $p(|x|)$ a polynomial function.

Define $M(x) = V(x, c)$ s.t. we loop over all strings c s.t. $|c| \leq p(|x|)$ and accept if $V(x, c)$ accepts or reject if $V(x, c)$ rejects for all c .

If $x \in L$ then there exists a c s.t. $p(|x|) \geq |c|$. Thus, our Turing machine $M(x)$ will loop through all possible strings c . Thus, eventually we will find a c s.t. $V(x, c)$ accepts thus $M(x)$ accepts.

If $x \notin L$ then there is no c s.t. $p(|x|) \geq |c|$ s.t. $V(x, c)$ will accept. Thus, $M(x)$ will reject. Thus, M is a decider for L . Thus, NP is a subset of decidable languages. ■

(c)

Prove or disprove: All NP-hard languages are decidable.

(A disproof would consist of an NP-hard language that is also undecidable.)

Proof.

False consider L_{Halt} . This is NP-Hard by proven on discussion worksheet complexity and undecidable. ■

Problem 3: Verifying Y's verifiers.

Recall that a *multiset* S is just like a set, but it can have duplicate elements. A sub-multiset $S^* \subseteq S$ also can have duplicate elements, as long as it does not have more copies of a particular element than S does.

$\text{SUBSETSUM} = \{(S, k) : S \text{ is a multiset of integers } \geq 0, \text{ and } \exists S' \subseteq S \text{ such that } \sum_{s \in S'} s = k\}.$

(a)

Require: A multiset of nonnegative integers $S = \{s_1, s_2, \dots, s_n\}$, an integer k , and a multiset of nonnegative integers $C = \{c_1, c_2, \dots, c_m\}$

```

1: function VERIFIERA( $S, k, C$ )
2:   if  $m > n$  then reject           ▷  $C$  can't be a submultiset of  $S$  in this case
3:    $sum \leftarrow 0$ 
4:   for  $c \in C$  do                     ▷ iterate over elements (integers) in  $C$ 
5:     if  $c \notin S$  then reject
6:     else  $sum \leftarrow sum + c$ 
7:   if  $sum = k$  then accept
8:   reject

```

Correctness: For correctness,

$$\begin{aligned}
 (S, k) \in \text{SUBSETSUM} &\iff \text{there exists a submultiset } C \subseteq S \text{ s.t. } \sum_{c \in C} c = k \\
 &\iff \text{VERIFIERA}(S, k, C) \text{ accepts}
 \end{aligned}$$

Runtime: After ruling out the possibility that $m > n$ in line 2, the for loop will iterate over at most n elements, while other lines take constant time, the overall runtime is $O(n)$, which is polynomial with respect to the input size.

Explain why VERIFIERA does not show that $\text{SUBSETSUM} \in \text{NP}$. You may use an example to support your reasoning. You do not have to fix the verifier, though it is a good exercise.

Proof.

VERIFIERA does not work since it does not account for if C has more elements of one type than S . For example if $S = \{1, 100, 100, 100, 100\}$, $k = 5$, and $C = \{1, 1, 1, 1, 1\}$. Then $V(S, k, C)$ would accept since $1 \in S$. Thus, $sum \leftarrow 5 = k$, but S cannot sum to 5. ■

(b)

Require: A multiset of nonnegative integers $S = \{s_1, s_2, \dots, s_n\}$, an integer k , and a collection of nonnegative integers multisets $C = \{C_1, C_2, \dots, C_m\}$

```

1: function VERIFIERB( $S, k, C$ )
2:   for  $C_i \in C$  do                                ▷ Iterate over multisets in  $C$ 
3:     Confirm that  $C_i \subseteq S$ 
4:     if  $\sum_{c \in C_i} c = k$  then accept
5:   reject

```

Correctness: For correctness,

$$\begin{aligned}
 (S, k) \in \text{SUBSETSUM} &\iff \text{there exists a submultiset } C_i \subseteq S \text{ s.t. } \sum_{c \in C_i} c = k \\
 &\iff \text{there exists a collection } C \text{ that contains } C_i \\
 &\iff \text{VERIFIERB}(S, k, C) \text{ accepts}
 \end{aligned}$$

Runtime: In line 4, since C_i is confirmed to be a submultiset of S , $|C_i| \leq n$ and hence summing all items takes $O(n)$. Since there are m sets in C , the overall runtime is $O(mn)$, which is polynomial with respect to the input size.

Explain why VERIFIERB does not show that SUBSETSUM $\in$ NP. You may use an example to support your reasoning. You do not have to fix the verifier, though it is a good exercise.

Proof.

Consider $C = \{C_1, C_2\}$ s.t. $C_1 \subset S$ whose elements sum to k , and C_2 s.t. $|C_2| = |S|^4$. Thus, even though the certificate is correct $|C_2|$ is not a polynomial in $|S|$. ■

(c)

Require: A multiset of nonnegative integers $S = \{s_1, s_2, \dots, s_n\}$, an integer k , and an array of bits $C[1, \dots, n]$

```

1: function VERIFIERC( $S, k, C$ )
2:    $sum \leftarrow 0$ 
3:   for  $i = 1, \dots, n$  do
4:     if  $C[i] = 1$  then  $sum \leftarrow sum + s_i$ 
5:   if  $sum = k$  then accept
6:   reject

```

Y claims that Lambda's verifier is incorrect with the following counterexample:

Consider $S = \{1, 1, 2, 5\}$, $k = 7$, and $C = [0, 1, 0, 1]$. Clearly, $S \in \text{SUBSETSUM}$, but $1 + 5 = 6 \neq 7$ so the verifier rejects. Therefore, VERIFIERC must be incorrect!

Explain why Y's claim is unfounded.

This is unfounded since there exists $C = [0, 0, 1, 1]$ s.t. $V(S, 7, C)$ accepts. We cannot throw out a verifier if one C does not work we have to check all C of a certain size.

Problem 4: Understanding poly-time mapping reductions.

(a)

Prove or disprove: if $A \leq_p B$, then $A \leq_T B$.

(A disproof would consist of a pair of languages such that $A \leq_p B$ but $A \not\leq_T B$.)

Proof.

Assume $A \leq_p B$ then this means there exists a efficient computable $f : \Sigma^* \rightarrow \Sigma^*$ s.t.
 $x \in A \iff f(x) \in B$.

Let M_B be an oracle that decides B .

Define: $M_A(x) = M_B(f(x))$.

M_A halts on any input because f is efficiently computable, and M_B halts on every input.

Therefore,

$$\begin{aligned} x \in A &\iff f(x) \in B \\ &\iff M_B(f(x)) \text{ accepts} \\ &\iff M_A(x) \text{ accepts} \end{aligned}$$

Thus, M_A decides A . Thus, $A \leq_T B$. ■

(b)

Prove or disprove: if $A \leq_T B$, then $A \leq_p B$.

(A disproof would consist of a pair of languages such that $A \leq_T B$ but $A \not\leq_p B$.)

Proof.

False

Consider $A = \emptyset$ and $B = \Sigma^*$

First since A is decidable we have that $\emptyset \leq_T \Sigma^*$

Assume for contradiction that $\emptyset \leq_p \Sigma^*$ thus there exists an efficiently computable f s.t. $x \in \emptyset \iff f(x) \in \Sigma^*$

Since there is no $x \in \emptyset$ this implies that $\forall x \in \Sigma^*$ that $f(x) \notin \Sigma^*$. This is a contradiction thus, $\emptyset \not\leq_p \Sigma^*$ ■

(c)

Prove that PTMRs are *transitive*: if $A \leq_p B$ and $B \leq_p C$, then $A \leq_p C$.

Proof.

Assume $A \leq_p B$ and $B \leq_p C$.

Thus, there exists efficiently computable f, g s.t. $x \in A \iff f(x) \in B$ and $y \in B \iff g(y) \in C$.

Thus, $g \circ f$ is efficient computable since $y = f(x)$ can be computed in $O(|x|^c)$ for some constant c . Since writing down is part of the output time we know that $|y| = O(|x|^c)$.

Thus, $g(y)$ can be computed in $O(|y|^d)$ for some constant d .

Thus, $g \circ f(x)$ can be computed in $O(|x|^c) + O(|f(x)|^d) = O(|x|^c + |x|^{cd})$ which is a polynomial in $|x|$.

Thus, $x \in A \iff f(x) \in B \iff g(f(x)) \in C$.

Therefore, $A \leq_p C$ ■

(d)

The languages Σ^* and $\emptyset$ are called the *trivial* languages, and all other languages are *non-trivial*. Prove that if $P = NP$, then *every* non-trivial language is NP-hard.

Be sure to state where your proof relies on non-triviality (because the trivial languages are *not* NP-hard, regardless of whether $P = NP$!).

Proof.

Assume $P = NP$. Let L be a non-trivial language.

Let $A \in NP = P$. Thus, there is a efficient turning machine M_A that decides A .

Since L is non-trivial this means there exists distinct $x, y \in \Sigma^*$ s.t. $x \in L$ and $y \notin L$.

Thus define $f(a) = x$ if $M_A(a)$ accepts else y .

Thus,

$$\begin{aligned} a \in A &\iff M_A(a) \text{ accepts} \\ &\iff f(a) = x \\ &\iff f(a) \in L. \end{aligned}$$

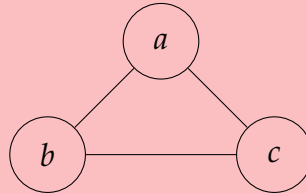
f is efficient computable since M_A is, and is correct by the proof above and the fact that M_A is correct and $x \neq y$.

Thus, $A \leq_p L$ for all $A \in NP$.

Thus, L is NP-Hard. ■

Problem 5: Dominating sets.

Consider an undirected graph $G = (V, E)$. A *dominating set* of a graph is a subset of vertices D such that every vertex in V is either in D or connected to some vertex/vertices in D . For example, consider the following graph:



The graph has a dominating set of size 1, for example, $D = \{a\}$ is a dominating set because $a \in D$ and both b and c are connected to a .

Notice the difference between a dominating set and a vertex cover: In the graph above, $D = \{a\}$ is *not* a vertex cover because it does not cover the edge (b, c) . However, $C = \{b, c\}$ is both a vertex cover and a dominating set.

Let G be an undirected graph and k be a nonnegative integer. Consider the following language:

$$\text{DOMINATING-SET} = \{(G, k) \mid G \text{ has a dominating set of size (at most) } k\}.$$

In this problem, you will show that DOMINATING-SET is NP-complete.

(a)

Prove that DOMINATING-SET $\in$ NP. Specifically, you need to

- (i) define a certificate and argue that it is polynomial in size with respect to the instance,
- (ii) give an efficient verifier for the language that takes in an instance and a certificate as input,
- (iii) give a correctness argument for the verifier, and
- (iv) give a runtime analysis for the verifier.

In your verifier, you may assume that the certificate is well-formed, so you do not have to check for its *data type* (e.g., a graph), but you still have to check for its *validity*.

Proof.

Given a graph $G = (V, E)$ with $|V| = n$ let our certificate C be a $C[1, \dots, n]$ array of bits. The certificate is exactly n bits long which is polynomial in $|G| > |V|$. Let V be a verifier s.t. (G, k) and C is its input.

1. Count:

- Counter the number of 1's in C .
- If the count is greater than k , then reject

2. Check:

- For each $v_i \in V$. Assume we have an order already build into the representation of V .
 - If $C[i] = 1$ then v_i is in the dominating set, so it is dominated.
 - If $C[i] = 0$ then check if there exists a neighbor v_j of v_i s.t. $C[j] = 1$. If there is no such neighbor reject.

3. Finally Accept.

Correctness:

If the verifier accepts, then every vertex v is either in the set or has a neighbor v_j with $C[j] = 1$. Thus, the set of vertices corresponding to 1's in C is a dominating set of G and its size is at most k .

If G has a dominating set of D of size at most k , then one can construct a certificate C by setting $C[i] = 1$ if $v_i \in D$ and $C[i] = 0$ if $v_i \notin D$.

Runtime:

First we count the n -bit array which takes $O(n) = O(|V|)$.

Then for each vertex $v \in V$, we check its neighbors. If the degree of v is d_v , then the total time for all vertices is $O(\sum_{v \in V} d_v) = O(2|E|) = O(|E|)$.

Thus, the total running time is $O(|V| + |E|)$ which is polynomial in the size of the input graph. ■

(b)

Since DOMINATING-SET has some resemblance to VERTEX-COVER, a good source of inspiration is the $3SAT \leq_p VERTEX-COVER$ reduction from lecture.

We modify the reduction as follows: Given a 3SAT instance (a 3CNF formula) ϕ , we output $f(\phi) = (G, k)$ where the “budget” $k = n$ is the number of variables in ϕ , and graph G is constructed as follows.

- For each variable x_i in ϕ we create a “variable gadget” consisting of **four** vertices labeled T_i (for true), F_i (for false), α_i , and β_i , along with edges (T_i, F_i) , (α_i, T_i) , (α_i, F_i) , (β_i, T_i) , and (β_i, F_i) .
- For each clause C_j in ϕ we create a **single** vertex labeled C_j , and introduce an edge between C_j and T_i if the literal x_i appears in C_j , and an edge between C_j and F_i if the literal $\bar{x}_i$ appears in C_j .

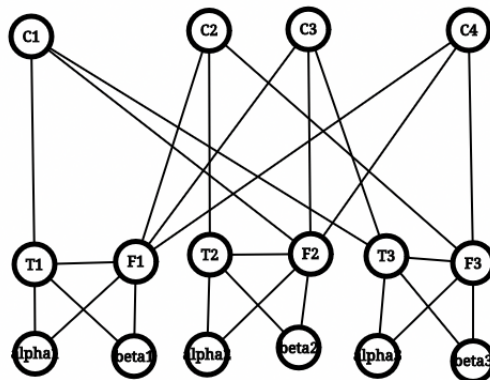
This reduction is correct, which you will prove in Part C.

Now, consider the following “yes” instance in 3SAT:

$$\phi = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

Determine $f(\phi) = (G, k)$, i.e., give the graph output by the reduction given ϕ and specify what k should be. Then, show that $\phi \in 3SAT$ by giving a dominating set of G that has (at most) k vertices (if there are multiple, just give any *one* of them).

Proof.



$$f(\phi) = (G, 3)$$

Consider $D = \{T_1, F_2, F_3\}$ this is a dominating set, and this confirms that $\phi \in 3SAT$

since if we plug in $x_1 = T$, $x_2 = F$, and $x_3 = F$ we get that ϕ is true. ■

(c)

Prove that the reduction in Part b is correct, i.e., show that

$$\phi \in 3SAT \iff f(\phi) = (G, k) \in \text{DOMINATING-SET}.$$

Moreover, the reduction takes polynomial time in the size of the input formula, because it creates $O(1)$ vertices and edges per vertex and clause. Since 3SAT is NP-hard, we can conclude that DOMINATING-SET is NP-hard.

Proof.

Let $\phi \in 3SAT$ then let τ be a truth assignment for ϕ .

If $\tau(x_i) = \text{true}$ then choose the vertex T_i If $\tau(x_i) = \text{false}$ then choose the F_i

Let D be the set of vertices we have chosen. Since we have picked exactly one vertex per variable we have that $|D| = k$.

Dominance of Gadgets:

In each gadget, the chosen vertex either T_i, F_i is adjacent to the other vertex in the pair T_i, F_i and to both α_i, β_i .

Therefore, all gadgets are dominated.

Dominate of Clause:

For each C_j in ϕ , consider the vertex in G .

Since τ satisfies ϕ , this means that each clause has one literal that is true.

Thus, if x_i is in C_j and $\tau(x_i) = \text{true}$, then C_j is adjacent to $T_i \in D$.

If $\bar{x}_i$ is in C_j and $\tau(x_i) = \text{false}$, then C_j is adjacent to $F_i \in D$.

Thus, every C_j is adjacent to some vertex in D .

Thus, D is a dominating set of size k .

Let $(G, k) \in \text{DOMINATING-SET}$ s.t. it has the structure defined by f .

Let D be a dominating set with size at most k .

Since α_i, β_i are only connected to T_i, F_i this implies that T_i or F_i or $(\alpha_i \text{ and } \beta_i) \in D$ and C_j are only connected to either T_i or F_i but not both

Since there are k gadgets this implies D has one vertex from each gadget. Since if not we would not dominate α_i or β_i . Thus, $|D| = k$, and has either T_i or F_i but not both from each gadget to dominate G .

Now define τ s.t. $\tau(x_i) = \text{true}$ if $T_i \in D$ and false otherwise. This is well-defined since we know that either T_i or F_i are in D .

For each clause vertex C_j it must be dominated. By construction each C_j is only connected to the vertices corresponding to literals in ϕ .

Since it is dominated we know it is adjacent to vertices in D . This means that the corresponding literal is set to true under τ .

Thus, every clause has at least one true literal under τ , so τ satisfies ϕ .

Therefore, $\phi \in 3\text{SAT} \iff f(\phi) \in \text{DOMINATING-SET}$

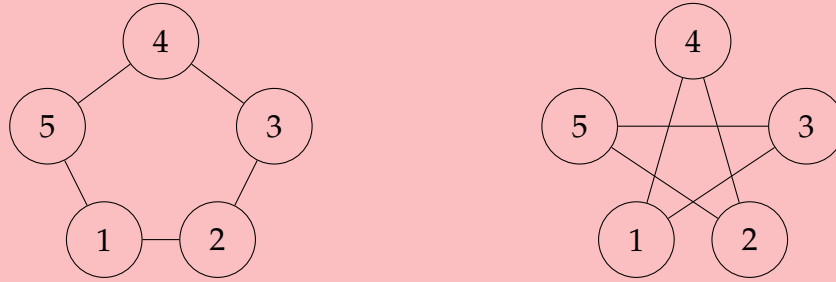
Complexity:

For each variable, we create 4 vertices and 5 edges. For each clause we create 1 vertex and at most 3 edges. Thus, given that $|\phi| = O(n + m)$ where n is the number of variables and m is the number of clauses. The total number of edges and vertices is $O(n + m)$. Thus, the reduction runs in polynomial time with respect to the size of ϕ .

Therefore, $3\text{SAT} \leq_p \text{DOMINATING-SET}$ ■

Problem 6: Subgraph isomorphism.

For two graphs $G = (V, E)$ and $H = (W, F)$, an *isomorphism* between them is a *bijection* $f : V \rightarrow W$ such that $(u, v) \in E$ if and only if $(f(u), f(v)) \in F$. Two graphs are said to be *isomorphic* if there exists an isomorphism between them. For example, the following two graphs are isomorphic, by the bijection f where $f(1) = 1$, $f(2) = 3$, $f(3) = 5$, $f(4) = 2$, and $f(5) = 4$.



Recall that a subgraph $G' = (V', E')$ of $G = (V, E)$ is a graph such that $V' \subseteq V$ and $E' \subseteq E$. Let G and H be undirected graphs. Consider the following language

$$\text{SUBGRAPH-ISOMORPHISM} = \{(\langle G \rangle, \langle H \rangle) : G \text{ has a subgraph } G' \text{ that is isomorphic to } H\}.$$

In this problem, we will prove that SUBGRAPH-ISOMORPHISM is NP-complete.

(a)

Prove that SUBGRAPH-ISOMORPHISM $\in$ NP.

Proof.

Let the certificate C be an array of vertices of V with length $|W|$. This is polynomial with respect to $(\langle G \rangle, \langle H \rangle)$ since it is just a subset of the vertices. Let V be a verifier s.t. $(\langle G \rangle, \langle H \rangle)$ and $\{(\langle w \rangle, \langle v \rangle)\}$ as input.

1. Check injection:

- Check that for every $C[i]$ and $C[j]$ s.t. $i \neq j$ that $C[i] \neq C[j]$
- If $C[i] = C[j]$ for $i \neq j$ then reject.

2. Check edges

- For every edge $(w_i, w_j) \in F$ check that $(C[i], C[j]) \in E$
- If not reject

Then accept.

Correctness:

If the verifier accepts, then the ordered pairs can be formed into a function f s.t. $f(w_i) = C[i]$. In step one we check it was an injection, and in step two we check it preserved edges thus $f(W) \cong H$, and so $\{\langle G \rangle, \langle H \rangle\} \in \text{SUBGRAPH-ISOMORPHISM}$

If there exists a subgraph G' that is isomorphic to H then we know there is an isomorphism from $W \rightarrow G'$. Let $C[i] = f(w_i)$. Thus, since f was an injection we know it will pass check 1, and since it is an isomorphism we know it will check step 2. Thus, V will accept the certificate C .

Runtime:

Step 1 will take $O(|W|^2)$ time to check all pairs in the C array.

Step 2 will take $O(|F|)$ steps to verify the edge-preservation.

Since both $|W|$ and $|F|$ are bounded by the input we know the overall verification runs in polynomial time with respect to the input. ■

(b)

Prove that $\text{CLIQUE} \leq_p \text{SUBGRAPH-ISOMORPHISM}$. Specifically, you need to

- give a PTMR from CLIQUE to $\text{SUBGRAPH-ISOMORPHISM}$,
- give a correctness argument for the reduction, and
- give a runtime analysis for the reduction.

Since CLIQUE is NP-hard, we can conclude that $\text{SUBGRAPH-ISOMORPHISM}$ is NP-hard.

Proof.

Define $R : \sigma^* \rightarrow \sigma^*$ s.t.

$$R(\langle G \rangle, \langle k \rangle) = (\langle G \rangle, \langle K_k \rangle)$$

To perform this algorithm all you have to do is construct K_k which takes $O(k^2)$ time since we create k vertices and $\binom{k}{2}$ edges.

Then we copy G which takes $O(|G|)$.

If G has a clique of size k , then there exists a set $V' \subset V$ with $|V'| = k$ s.t. the subgraph induced by V' is isomorphic to K_k thus $(G, K_k) \in \text{SUBGRAPH-ISOMORPHISM}$

If $(G, K_k) \in \text{SUBGRAPH-ISOMORPHISM}$, then there is a subgraph G' of G that is isomorphic to K_k . By definition, this means that G' is a clique of size k , so $(G, k) \in \text{CLIQUE}$

Runtime:

Like I said above it takes $O(|G| + k^2)$ times to copy over G and create K_k thus this is polynomial of the size of the input $|(G, k)| \approx |G| + |k|$.

Therefore, $\text{CLIQUE} \leq_p \text{SUBGRAPH-ISOMORPHISM}$ ■

Problem 7: ig independent sets.

Recall that an *independent set* of an undirected graph $G = (V, E)$ is a subset $I \subseteq V$ such that no two (distinct) vertices in I have an edge between them.

In lecture, we proved that the following language is NP-complete.

$$\text{IS} = \{(G, k) : G = (V, E) \text{ has an independent set of size (at least) } k\}.$$

In this problem, we will explore the following variant of IS.

$$\text{BIG-IS} = \{G : G = (V, E) \text{ has an independent set of size (at least) } |V| - 1\}.$$

Either prove that $\text{BIG-IS} \in \text{P}$ (by giving a polynomial-time algorithm that decides BIG-IS) or that BIG-IS is NP-hard (by showing that $\text{IS} \leq_p \text{BIG-IS}$).

Proof.

This checks to see if there is an independent set of size $|V| - 1$

Algorithm 1 BIG-IS

Require: An undirected graph $G = (V, E)$.

Ensure: **Accept** if G has an independent set of size $|V| - 1$, **Reject** otherwise.

```

1: if  $|E| = 0$  then
2:   return Accept
3: for each vertex  $v \in V$  do
4:    $valid \leftarrow \text{true}$ 
5:   for each edge  $(u, w) \in E$  do
6:     if  $u \neq v$  and  $w \neq v$  then
7:        $valid \leftarrow \text{false}$ 
8:       break from inner loop
9:   if  $valid$  is true then
10:    return Accept  $\triangleright V \setminus \{v\}$  is an independent set of size  $|V| - 1$ 
11: return Reject

```

Correctness:

This halts on every input since we do two for loops but since G is finite we will end.

If G is totally disconnected then G trivially has an independent set I of size at least

$|V| - 1$, so this implies that $|E| = 0$ thus we accept on line 2.

If the algorithm accepts on line two then this implies that $|E| = 0$ thus G is totally disconnected and trivially has an independent set I of size at least $|V| - 1$.

If G is not totally disconnected and has an independent set I of size $|V| - 1$.

Let $v \in V \setminus I$

Assume for contradiction that there exists an edge $(u, w) \in E$ s.t. both $u, w \in I$ and $u \neq v \neq w$. Since I is an independent set, no two distinct vertices in I can be adjacent, and so such an edge cannot exist.

Thus, when we are on the loop for vertex v , we will never hit step 7 in our algorithm thus *valid* will always be true. Thus, we will accept G .

If our algorithm accepts on line 10 then this means that for some $v \in V$ every edge $(u, w) \in E$ there was no pair s.t. $u = v$ or $v = w$. This implies that no two vertices in $V \setminus \{v\}$ have an edge between them. Thus, $I = V \setminus \{v\}$ is an independent set.

Thus, our algorithm decides BIG-IS

Runtime:

The algorithm loops $|V|$ times then it loops $|E|$ times. The rest of the operations are $O(1)$.

Thus overall we take $O(|V||E|)$ time which is a polynomial on the input. ■

Problem 8: A ticket to \$1 million (optional extra credit)

The complexity class coNP is defined as the set of languages whose *complement* languages are in NP :

$$\text{coNP} = \{\bar{L} : L \in \text{NP}\}$$

Note that $\text{coNP} \neq \overline{\text{NP}}$, which is the class of languages that are not in NP .

In this problem, we will explore the complexity class *difference polynomial-time*, denoted as D^{P} , which contains all languages that can be expressed as the intersection of an NP language and a coNP language:

$$\text{D}^{\text{P}} = \{L : L = A \cap B \text{ for some } A \in \text{NP} \text{ and } B \in \text{coNP}\},$$

or equivalently,

$$\text{D}^{\text{P}} = \{L : L = A \cap \bar{B} \text{ for some } A, B \in \text{NP}\}.$$

Note that $\text{D}^{\text{P}} \neq \text{NP} \cap \text{coNP}$, which is the set of languages that are both in NP and in coNP .

(a)

Let ϕ_1 and ϕ_2 be 3CNF formulas. Consider the following language

$$3\text{SAT-3UNSAT} = \{(\phi_1, \phi_2) : \phi_1 \text{ is satisfiable but } \phi_2 \text{ is not}\}.$$

Show that 3SAT-3UNSAT is D^{P} -complete. In particular,

- (i) briefly explain why $3\text{SAT-3UNSAT} \in \text{D}^{\text{P}}$, and
- (ii) prove that 3SAT-3UNSAT is D^{P} -hard, i.e., $L \leq_p 3\text{SAT-3UNSAT}$ for all $L \in \text{D}^{\text{P}}$.

Proof.

- $L_1 = \{(\phi_1, \phi_2) \mid \phi_1 \text{ is satisfiable}\} = 3\text{SAT} \in \text{NP}$ This is in NP because determining if a given 3CNF formula is satisfiable is exactly 3SAT
 - $L_2 = \{(\phi_1, \phi_2) \mid \phi_2 \text{ is unsatisfiable}\} = \overline{3\text{SAT}} \in \text{coNP}$ This is in coNP because determining if a given 3CNF formula is unsatisfiable is exactly the opposite of checking if it is satisfiable thus it is the complement of 3SAT.
- Thus, $3\text{SAT-3UNSAT} = L_1 \cap L_2 \in \text{D}^{\text{P}}$
- Let $L = A \cap \bar{B}$ for some $A, B \in \text{NP}$.

Since $A, B \in \text{NP}$ and 3SAT is NP-hard. We know there are efficiently computable functions f, g s.t.

$$x \in A \iff f(x) \in \text{3SAT}$$

$$x \in B \iff g(x) \in \text{3SAT}$$

Define $R(x) = (f(x), g(x))$

Correctness:

$$\begin{aligned} x \in L &\iff x \in A \text{ and } x \notin B \\ &\iff f(x) \in \text{3SAT} \text{ and } g(x) \notin \text{3SAT} \\ &\iff (f(x), g(x)) \in \text{3SAT-3UNSAT} \end{aligned}$$

Runtime:

Since f, g are efficient computable this means $f(x)$ runs in $O(|x|^d)$ time and $g(x)$ runs in $O(|x|^c)$ time. Thus, $R(x)$ runs in $O(|x|^d + |x|^c)$. Thus, R is efficiently computable. Therefore, 3SAT-3UNSAT is D^{P} -complete. ■

(b)

Show that if $\text{D}^{\text{P}} \neq \text{NP}$, then $\text{P} \neq \text{NP}$.

Proof.

Assume $\text{D}^{\text{P}} \neq \text{NP}$. Assume for contradiction that $\text{P} = \text{NP}$.

Since $\text{NP} = \text{P}$ is closed under complement we have $\text{coNP} = \text{P}$.

Thus, $\text{NP} = \text{P} = \text{coNP}$.

Let $L \in \text{D}^{\text{P}}$. Thus, $L = A \cap \overline{B}$ for $A, B \in \text{NP}$. Since $\text{NP} = \text{P} = \text{coNP}$ and that P is closed under intersection this implies that $L \in \text{P}$.

Thus, $\text{D}^{\text{P}} \subset \text{P} = \text{NP}$

Let $L \in \text{NP} = \text{coNP}$ consider $L = L \cap L$. Thus, $L \in \text{D}^{\text{P}}$.

Therefore, $\text{D}^{\text{P}} = \text{NP}$ which is a contradiction. Thus, $\text{P} \neq \text{NP}$. ■